深圳大学
Shenzhen University

互联网编程

第5章

Url和URI

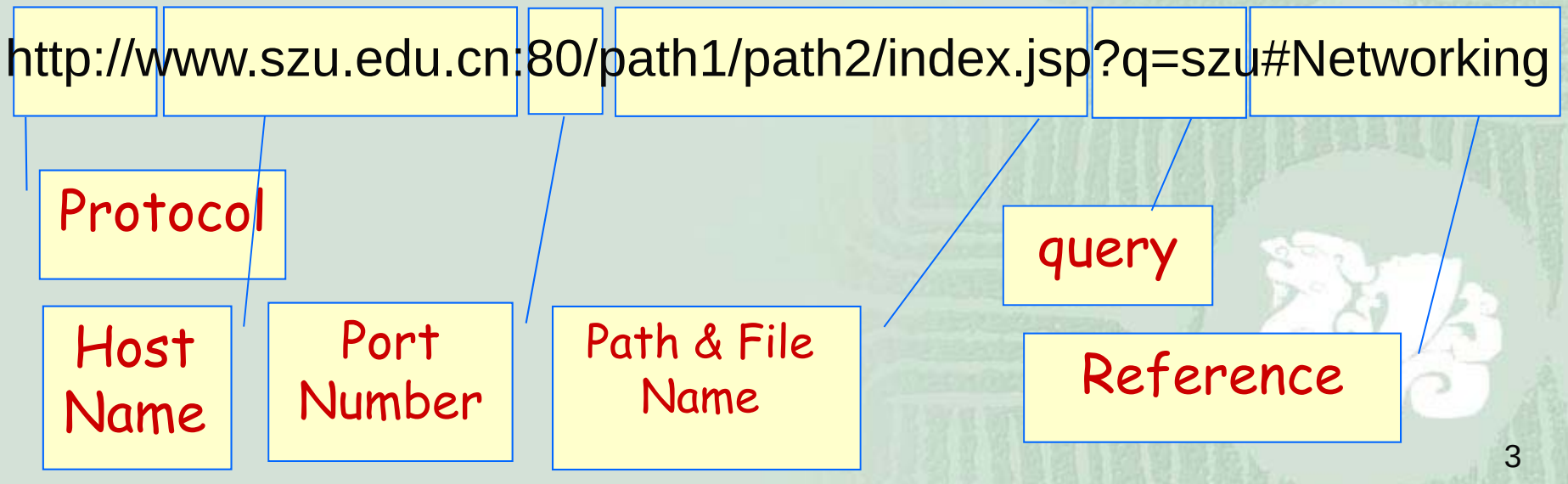
深圳大学计算机与软件学院

主要内容

- 1. URI与URL概念
- 2. URL类
- 3. URI类
- 4. URLEncoder类与URLDecoder类
- 5. 访问服务器端的GET方法
- 6. Authenticator类和PasswordAuthenticator类

1.URI&URL概念

- URI：Uniform Resource Identifier
- URL：Uniform Resource Locator，用于在程序中指定网络数据源
- URL标识资源，提供资源位置信息，告诉浏览器或互联网应用程序一个文档的如下信息：



绝对URL，相对URL

- 完整指定的URL称为**绝对URL**
- http://www.szu.edu.cn/2014/overview.html
- 不完整指定的、继承了其父文档（该url所在的文档)的协议、主机名和路径的URL,称为相对URL

在浏览http://www.szu.edu.cn/2014/overview.html时单击链接：`<a href="planning.html">校园规划</a>`

实际是访问 `http://www.szu.edu.cn/2014/planning.html`

- planning.html
- 相对URL优点：
  - 减少文字录入
  - 便于网站迁移

相对URL

- http://www.szu.edu.cn/2014/overview.html
  - 相对链接以/开头，表示相对于根目录而不是当前文件
  - 问：浏览器加载 相对链接 /2016/planning.html 时，实际加载的是？

http://www.szu.edu.cn/2016/planning.html

2.URL类

- java.net.URL类是用java对URL的封装和定义
- 使用了策略(strategy)设计模式
- 拓展思考：什么是策略模式，如何应用？

- Final 类型，一个URL对象一旦创建后不可修改
- JVM支持的协议有http、https、jar、ftp等

创建URL对象（6个构造方法）

Constructors

Constructor and Description
`URL(String spec)` Creates a URL object from the String representation.
`URL(String protocol, String host, int port, String file)` Creates a URL object from the specified protocol, host, port number, and file.
`URL(String protocol, String host, int port, String file, URLStreamHandler handler)` Creates a URL object from the specified protocol, host, port number, file, and handler.
`URL(String protocol, String host, String file)` Creates a URL from the specified protocol name, host name, and file name.
`URL(URL context, String spec)` Creates a URL by parsing the given spec within a specified context.
`URL(URL context, String spec, URLStreamHandler handler)` Creates a URL by parsing the given spec with the specified handler within a specified context.

URL类：协议支持

```
private static void testProtocol(String url) {
  try {
    URL u = new URL(url);
    System.out.println(u.getProtocol() + " is supported");
  } catch (MalformedURLException ex) {
    String protocol = url.substring(0, url.indexOf(':'));
    System.out.println(protocol + " is not supported");
  }
}
```

例子EX5-1：查看虚拟机支持哪些协议？

提示：查看解释代码ch5.ProtocolTester.java

ProtocolTester.java

```java
import java.net.*;
public class ProtocolTester {
  public static void main(String[] args) {
    // hypertext transfer protocol
    testProtocol("http://www.adc.org");
    // secure http
    testProtocol("https://www.amazon.com/exec/obidos/order2/");
    // file transfer protocol
    testProtocol("ftp://ibiblio.org/pub/languages/java/javafaq/");
    // Simple Mail Transfer Protocol
    testProtocol("mailto:elharo@ibiblio.org");
    // telnet
    testProtocol("telnet://dibner.poly.edu/");
    // local file access
    testProtocol("file:///etc/passwd");
    // gopher
    testProtocol("gopher://gopher.anc.org.za/");
    // Lightweight Directory Access Protocol
    testProtocol(
        "ldap://ldap.itd.umich.edu/o=University%20of%20Michigan,c=US?postalAddress");
    // JAR
    testProtocol(
        "jar:http://cafeaulait.org/books/javaio/ioexamples/javaio.jar!"
         + "/com/macfaq/io/StreamCopier.class");
    // NFS, Network File System
    testProtocol("nfs://utopia.poly.edu/usr/tmp/");
```

ProtocolTester.java

```
28      // a custom protocol for JDBC
29      testProtocol("jdbc:mysql://luna.ibiblio.org:3306/NEWS");
30      // rmi, a custom protocol for remote method invocation
31      testProtocol("rmi://ibiblio.org/RenderEngine");
32      // custom protocols for HotJava
33      testProtocol("doc:/UsersGuide/release.html");
34      testProtocol("netdoc:/UsersGuide/release.html");
35      testProtocol("systemresource://www.adc.org/+/index.html");
36      testProtocol("verbatim:http://www.adc.org/");
37    }
38
39    private static void testProtocol(String url) {
40      try {
41        URL u = new URL(url);
42        System.out.println(u.getProtocol() + " is supported");
43      } catch (MalformedURLException ex) {
44        String protocol = url.substring(0, url.indexOf(':'));
45        System.out.println(protocol + " is not supported");
46      }
47    }
48  }
49
```

EX5-1运行结果

```
<terminated> ProtocolTester [Java Application] C:\Progr
http is supported
https is supported
ftp is supported
mailto is supported
telnet is not supported
file is supported
gopher is supported
ldap is not supported
jar is supported
nfs is not supported
jdbc is not supported
rmi is not supported
doc is not supported
netdoc is supported
systemresource is not supported
verbatim is not supported
```

字符串构造URL对象

定义：

```
public URL(String url) throws MalformedURLException
```

例子：

```
try {
URL u = new URL("http://www.szu.edu.cn/");
}
catch (MalformedURLException ex) {
    System.err.println(ex);
}
```

分块构造URL对象

- 通过指定协议、主机名和文件来构建一个URL，定义：

```
public URL(String protocol, String hostname, String file)throws MalformedURLException
```

- 例子：

注意：勿遗漏file参数前的/

```
try {
    URL u = new URL("http", "www.eff.org", "/blueribbon.html#intro");
}
catch (MalformedURLException ex) {
    throw new RuntimeException("shouldn't happen; all VMs recognize http");
}
```

#隔开的intro 是片段标识符

代码片段效果：创建一个URL对象，指向 http://www.eff.org/blueribbon.html#intro，使用HTTP的默认端口80

构造相对URL

- 根据相对URL和基础URL构建一个绝对URL,定义：

```
public URL(URL base, String relative)
throws MalformedURLException
```

- 例子：

```
try {
URL u1 = new URL("http://www.szu.edu.cn/a/b.html");
URL u2 = new URL(u1, "mailinglists.html");
}
catch (MalformedURLException ex) {
    System.err.println(ex);
}
```

代码片段效果：创建一个U1对象，指向 http://www.szu.edu.cn/a/b.html ，将文件名从u1的路径中删除，追加上新文件名mailinglists.html，得到u2对象，指向 http://www.szu.edu.cn/a/mailinglists.html

思考

- 试从程序输入输出处理数据的角度思考并解释一下，从PC浏览器地址栏输入一个url，到可以在浏览器中看到这个网页内容，什么程序从哪里获得了什么数据？处理了什么数据？

从URL获取数据

- URL类的获取数据方法：
- 获得InputStream
  - `public InputStream openStream()`
- 获得URLConnection对象
  - `public URLConnection openConnection()`
  - `public URLConnection openConnection(Proxy proxy)`
- 获得内容对象如String或Image
  - `public Object getContent()`
  - `public Object getContent(Class[] classes)`

从URL获取数据：openStream（）方法

- openStream方法连接到URL所引用的资源，在客户端和服务器之间建立可靠连接，返回一个InputStream，以读取数据。

```
try {
    URL u = new URL("http://www.szu.edu.cn");
    InputStream in = u.openStream();
    int c;
    while ((c = in.read()) != -1)
        System.out.write(c);
    in.close();
}catch (IOException ex) {
    System.err.println(ex);
}
```

例子5-2：下载一个Web页面

- 例子EX5-2：下载一个Web页面，在控制台输出该页面的原始HTML。

- 思路：从命令行读取一个URL（http://www.oreilly.com），从这个URL打开一个InputStream，将此InputStream串链到默认编码方式的InputStreamReader，使用其read()方法从文件读取连续的字符，将字符在控制台显示输出。

  - 提示：查看解释代码ch5.SourceViewer.java

EX5-2运行结果

运行：运行SourceViewer，并输入参数http://www.oreilly.com，运行结果如下

```
<terminated> SourceViewer [Java Application] C:\Program Files\Java\jdk1.7.0_07\bin\javaw.exe (2017-4-
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="https://www.oreilly.com/">here</a>.</p>
</body></html>
```

测试练习并观察：若将参数改为http://www.szu.edu.cn，请观察控制台运行结果，并对比从浏览器输入该URL的返回结果。

SourceViewer.java

```java
2  import java.io.*;
3  import java.net.*;
4  public class SourceViewer {
5    public static void main (String[] args) {
6      if (args.length > 0) {
7        InputStream in = null;
8        try {
9          // Open the URL for reading
10         URL u = new URL(args[0]);
11         in = u.openStream();
12         // buffer the input to increase performance
13         in = new BufferedInputStream(in);
14         // chain the InputStream to a Reader
15         Reader r = new InputStreamReader(in);
16         int c;
17         while ((c = r.read()) != -1) {
18           System.out.print((char) c);
19         }
20       } catch (MalformedURLException ex) {
21         System.err.println(args[0] + " is not a parseable URL");
22       } catch (IOException ex) {
23         System.err.println(ex);
24       } finally {
25         if (in != null) {
26           try {
27             in.close();
```

SourceViewer.java

```java
29                 // ignore
30             }
31           }
32         }
33       }
34     }
35   }
36
```

思考

- 这程序有没有问题？问题在哪里？

- **程序不可靠：**
  - 1.轻率地假定目标URL指向的是文本，但可能指向的是GIF或JPEG图像、MP3声音文件等。
  - 2.即使是文本，远程主机的文档编码方式也可能与客户端系统的默认编码方式不同，使用的是不同的默认字符集。
  - 3.你在编程时是不是也忽略了这些问题，写了不靠谱的程序？

编码方式的指定

- 通常一个页面或网络传输文件，如果使用了与ASCALL完全是不同的字符集,会在头部指明。

例1，一个HTML文件在META标记中指明采用的是Big-5编码方式：

<meta http-equiv="content-type" content="text/html; charset=big5">

例2,一个XML文件，在头部声明其编码方式：

<?xml version="1.0" encoding="Big5"?>

课后补充学习内容： html、xml文档格式

从URL对象获取数据：openConnection（）方法

- 该方法为指定的URL打开一个socket，并返回一个URLConnection对象，
- 通过此对象可以访问协议指定的所有元数据和数据。

```
try {
URL u = new URL("http://www.szu.edu.cn/");
    try {
        URLConnection uc = u.openConnection();
        InputStream in = uc.getInputStream();
        // read from the connection...
    } catch (IOException ex) {
        System.err.println(ex);
    }
} catch (MalformedURLException ex) {
   System.err.println(ex);
}
```

如果希望在程序中能与服务器直接通信，访问服务器发送的所有数据，应当使用些方法。可以访问服务器发送的原始文档（HTML/纯文本/二进制图像数据）和协议指定的所有元数据（HTTP头部,原始html）。

从URL对象获取数据：openConnection

- 可访问服务器发送的所有数据
  - 如HTTP头信息
- 可以向服务器发送数据
  - Email
  - 提交表单
- openConnection(Proxy proxy) 方法的重载版本
  - 可设置代理服务器，指定通过哪个代理服务器传递连接

从URL对象获取数据：getContent（）方法

- 该方法获取由URL引用 的数据对象，程序可尝试由它建立某种类型的数据对象。
- 用法：

```
URL u = new URL("http://mesola.obspm.fr/");
Object o = u.getContent();
//强制将对象o转换为特定类型
if(o instanceof URLImageSource){
    ...
```

该方法在从服务器获取的数据头部中查找content-type字段，如果服务器没有使用MIME头部或使用了一个陌生的content-type，该方法会返回某种InputStream,供程序读取数据。

```
public class ContentGetter {

  public static void main (String[] args) {

    if  (args.length > 0) {
      // Open the URL for reading
      try {
        URL u = new URL(args[0]);
        Object o = u.getContent();
        System.out.println("I got a " + o.getClass().getName());
      } catch (MalformedURLException ex) {
        System.err.println(args[0] + " is not a parseable URL");
      } catch (IOException ex) {
        System.err.println(ex);
      }
    }
  }
}
```

EX5-3运行结果

- 运行：运行ContentGetter，并输入参数http://www1.szu.edu.cn，运行结果如下

```
<terminated> ContentGetter [Java Application] C:\Program Files\Java\jdk1.7.0_07\bin\javaw.exe (2
I got a sun.net.www.protocol.http.HttpURLConnection$HttpInputStream
```

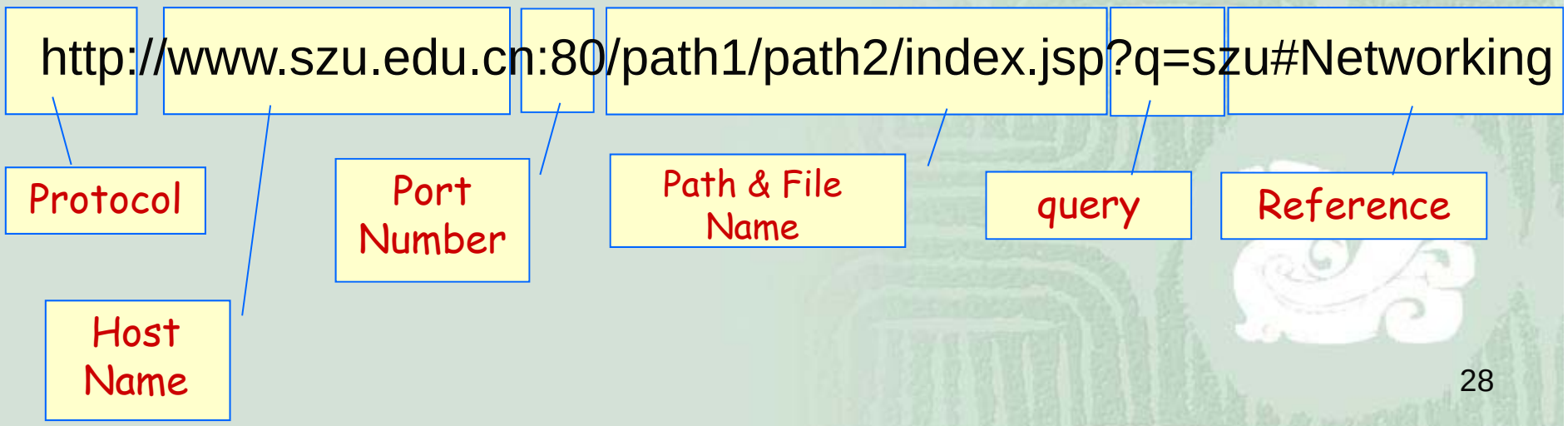
- 请尝试运行时输入参数URL指向一个图像文件，观察其结果。
- 请尝试运行时输入参数URL指向一个class文件，观察其结果。
- 请尝试运行时输入参数URL指向一个音频文件，观察其结果。

- 思考：使用getContent()方法：很难预测获得哪种对象（InputStream/ImageProducer/AudioClip），
- 需使用instanceof操作符进行检查。

分解URL

- URL的5部分
  - 模式（scheme)，即协议
  - 授权机构（authority）
  - 路径（path）
  - 片段标识符（fragment identifier/section /ref）
  - 查询字符串（query string）

Authority: 是//与端口号之间的内容，包括端口号。
www.szu.edu.cn:80



分解URL

- URL类提供了9个public方法读取URL信息
  - getFile(),
  - getHost(),
  - getPort(),
  - getProtocol(),
  - getRef(),
  - getQuery(),
  - getPath(),
  - getUserInfo(),
  - getAuthority()

  http://admin@www.blackstar.com:8080/

  授权机构admin@www.blackstar.com:8080

- public String getProtocol()
- 将url中的协议以字符串形式返回。

```
URL u = new URL("https://xkcd.com/a/");
System.out.println(u.getProtocol());
显示输出字符串"https“
```

- public String getHost()
- 将url中的主机以字符串形式返回。

```
URL u = new URL("https://xkcd.com/a/");
System.out.println(u.getHost());
显示输出字符串“xkcd.com“
```

- public int getPort()
- 返回-1或url中指定的端口号。

```
URL u = new URL("http://www.userfriencly.org/");
System.out.println(u.getPort());
显示输出端口号为-1
```

- public int getDefaultPort()
- 返回-1或url中协议使用的默认端口。

```
URL u = new URL(“ftp://ftp.userfriendly.org:8000/");
System.out.println(u.getDefaultPort());
显示输出端口号21
```

- public String getFile()
- 把url中从第一个斜线(/)一直到片段标识符#之前的字符当作文件部分返回。

```
URL u = new URL("http://www.szu.edu.cn/doc/index.jsp?q=szu#Networkin");
System.out.println(u.getFile());
```

显示输出为：/doc/index.jsp?q=szu

- public String getPath()
- 把url中路径和文件部分返回，不包括查询字符串。

```
URL u = new URL("http://www.szu.edu.cn/doc/index.jsp?q=szu#Networkin");
System.out.println(u.getPath());
```

显示输出为：/doc/index.jsp

- public String getRef()
- 返回null或url中的片段标识符部分。

```
URL u = new URL("http://www.szu.edu.cn/doc/index.jsp?q=szu#Networkin");
System.out.println(u.getRef());
```

显示输出为：Networkin

- public String getQuery()
- 返回null或url中的查询字符串。

```
URL u = new URL("http://www.szu.edu.cn/doc/index.jsp?q=szu#Networkin");
System.out.println(u.getQuery());
```

显示输出为：q=szu

- public String getUserInfo()
- 把url中协议后面双斜线(//)开始一直到@之前的字符当作用户信息返回。

```
URL u = new URL("ftp://mp3:secret@ftp.example.com:210/c%3a/stuff/mp3/");
System.out.println(u.getUserInfo());
```

显示输出为：mp3:secret

但在mailto类型url如maito:elharo@ibiblio.org中, elharo@ibiblio.org是路径。

- public String getAuthority()
- 把url中协议后面双斜线(//)开始一直到路径之前的字符当作授权机构信息返回。

```
URL u = new URL(" ftp://mp3:secret@ftp.example.com:210/c%3a/stuff/mp3/ ");
System.out.println(u.getAuthority());
```

显示输出为：mp3:secret@ftp.example.com:210

```
import java.net.*;
public class URLSplitter {
  public static void main(String args[]) {
    for (int i = 0; i < args.length; i++) {
      try { URL u = new URL(args[i]);
              System.out.println("The URL is " + u );
              System.out.println("The scheme is " + u.getProtocol());
              System.out.println("The user info is " + u.getUserInfo());
              String host = u.getHost();
              if (host != null) {   int atSign = host.indexOf('@');
                                     if (atSign != -1) host = host.substring(atSign+1);
                                    System.out.println("The host is " + host);
        } else {          System.out.println("The host is null.");    }
        System.out.println("The port is " + u.getPort());
        System.out.println("The path is " + u.getPath());
        System.out.println("The ref is " + u.getRef());
        System.out.println("The query string is " + u.getQuery());
      } catch (MalformedURLException ex) {
         System.err.println(args[i] + " is not a URL I understand.");  }
      System.out.println();
    }//for      }//main()      }//URLSplitter
```

```
<terminated> URLSplitter [Java Application] C:\Program Files\Java\jdk1.7.0_07\bin\javaw.exe (2017-5-4 下午4:14:43
The URL is ftp://mp3:mp3@202.38.48.14:21000/c%3a/
The scheme is ftp
The user info is mp3:mp3
The host is 202.38.48.14
The port is 21000
The path is /c%3a/
The ref is null
The query string is null

The URL is http://www.oreilly.com
The scheme is http
The user info is null
The host is www.oreilly.com
The port is -1
The path is
The ref is null
The query string is null

The URL is http://admin@ww.blackstar.com:8080/nywc/composition.html?category=piano
The scheme is http
The user info is admin
The host is ww.blackstar.com
The port is 8080
The path is /nywc/composition.html
The ref is null
The query string is category=piano
```

比较两个url

- 若想比较两个url是否一样，指向的资源是否一样，如何编程？
  - 使用URL类的equals()方法。
  - 或使用URL类的sameFile()方法。

- 这两个url(http://www.ibiblio.org和http://ibiblio.org)一样吗？
- 编程序来验证一下：Ex5-5 URLEquality.java

```
import java.net.*;
public class URLEquality {
  public static void main (String[] args) {
    try {
      URL www = new URL ("http://www.ibiblio.org/");
      URL ibiblio = new URL("http://ibiblio.org/");
      if (ibiblio.equals(www)) {
        System.out.println(ibiblio + " is the same as " + www);
      } else {
        System.out.println(ibiblio + " is not the same as " + www);
      }
    } catch (MalformedURLException ex) {
      System.err.println(ex);
    }
  }
}
```

```
<terminated> URLEquality [Java Application] C:\Program Files\Java\jdk1.7.0_07\bi
http://ibiblio.org/ is the same as http://www.ibiblio.org/
```

比较两个url

- 明明两个url字符串不一样的嘛，为啥比较结果是一样？
- 因URL类的equals()方法会尝试用DNS解析url中的主机，判断两个主机是否相同。
- 但不会具体比较两个url标识的资源。

- 例如：该方法的比较结果会返回
- http://www.oreilly.com与http://www.oreilly.com/index.html是不等的。
- http://www.oreilly.com与http://www.oreilly.com:80是不等的。
- 用sameFile()方法检查两个url是否指向相同资源。
- 例如：http://www.oreilly.com/index.html#p1 与 http://www.oreilly.com/index.html#q2
- 用sameFile()方法检查返回结果是： 两个是相等的，即指向相同的资源。
- 用equals()方法返回结果是 ：两个是不等的。
- 局限：sameFile()方法比较时不考虑片段标识符。

3.URI类

- URI：Uniform Resource Identifier
  - URL：Uniform Resource Locator
    - http://www.szu.edu.cn/szu.asp
  - URN：Uniform Resource Name
    - urn:isbn:0-486-27557-4
    - mailto:feiqiao@szu.edu.cn

- 虽URL可视作一种URI的抽象，但Java的URL类并不是URI的子类
- URL类与URI类皆是Object的子类
- Java.net.URI类

URL和URI的选用

- 编程实现网络数据获取功能时选用URL类
- 比如 想要下载一个url的内容。

- 编程用于解析和处理统一资源定位符相关的字符串时选用URI类。
- URI类无网络获取功能。
- 比如 想要表示一个XML命名空间。

URI类的构造方法

- 根据传入的字符串创建一个**URI**对象
- **public** URI(String uri) **throws** URISyntaxException
- 不检查协议，只检查传入的字符串要合uri语法规则。

```
URI voice = new URI("tel:+1-800-9988-9938");
URI web = new URI("http://www.a.com/a.jsp#id=hbc");
URI book = new URI("urn:isbn:1-565-92870-9");
```

URI格式：

协议：协议特定部分：片段

URI类的构造方法

- 根据传入的模式、模式特定部分、片段标识符、 字符串创建一个URI对象
- **public** URI(String scheme, String schemeSpecificPart, String fragment) **throws** URISyntaxException
- Scheme: uri的协议，如http、urn、tel等，必须以字母开头，由ASCAII字符、数字和三个标点符（+、-、.)组成。为null时，省略协议，创建 一个相对uri。
- Fragment: 为null时，表示忽略片段标识符。

```
URI absolute = new URI("http", "//www.ibiblio.org" , null);
URI relative = new URI(null, "/javafaq/index.shtml", "today");
```

URI各部分的获取

- scheme:scheme-specific-part:fragment
  - 协议：协议特定部分：片段

- 方法
  - `public String getScheme()`
  - `public String getSchemeSpecificPart()`
  - `public String getRawSchemeSpecificPart()`
  - `public String getFragment()`
  - `public String getRawFragment()`

  - `public boolean isOpaque()`

相对URI编程解析

```
URI absolute = new URI("http://www.example.com/");
URI relative = new URI("images/logo.png");
URI resolved = absolute.resolve(relative);
```

- resolved结果是：
- http://www.example.com/ images/logo.png

```
URI top = new URI("javafaq/books/");
URI resolved = top.resolve("jnp3/index.html");
```

- resolved结果是：
- javafaq/books/jnp3/index.html

4.URLEncoder类与URLDecoder类

- 为使各系统和平台都能正确理解和解析，URL中使用的字符必须来自ASCII的一个固定子集,即采用统一的URL字符编码
  - 大写字母A-Z
  - 小写字母a-z
  - 数字0-9
  - 标点符号字符-_.!~*'(,)
  - 标点符号字符/&?@#;$+=%
- URL中的非上述字符需要转为“%字节值”
  - %20：空格

URLEncoder类

- Java.net.URLEncoder类用于编程准备查询字符串，从而可与使用GET方法的服务器端程序通信

```
String encoded =
URLEncoder.encode("This*string*has*asterisks"
, "UTF-8");
```

- URLEncoder.encode()将字符串中的字符进行编码，使之符合URL字符规范

使用URLEncoder编码生成查询字符串

查询串1.
https://www.google.com/search?hl=en&as_q=Java&as_epq=I/O

- 将查询串1编码

```
String query =
URLEncoder.encode(https://www.google.com/search?hl=en&as_q=Java$as
_epq=I/O,"UTF-8");
System.out.println(query);
```

- 输出为：
  https%3A%2F%2Fwww.google.com%2Fsearch%3Fhl%3Den%26as_q%3DJava%26as_epq%3DI%2FO

存在盲目编码问题：URLEncoder.encode()会盲目地进行编码，不区分URL查询字符串中使用的特殊字符和需要编码的字符.

使用URLEncoder编码生成查询字符串

- 盲目编码问题的解决

```
String url = "https://www.google.com/search?";
url += URLEncoder.encode("hl", "UTF-8");
url += "=";
url += URLEncoder.encode("en", "UTF-8");
url += "&";
url += URLEncoder.encode("as_q", "UTF-8");
url += "=";
url += URLEncoder.encode("Java", "UTF-8");
url += "&";
url += URLEncoder.encode("as_epq", "UTF-8");
url += "=";
url += URLEncoder.encode("I/O", "UTF-8");
System.out.println(url);
```

输出为想要的输出：

```
https://www.google.com/search?hl=en&as_q=Java&as_epq=I/O
```

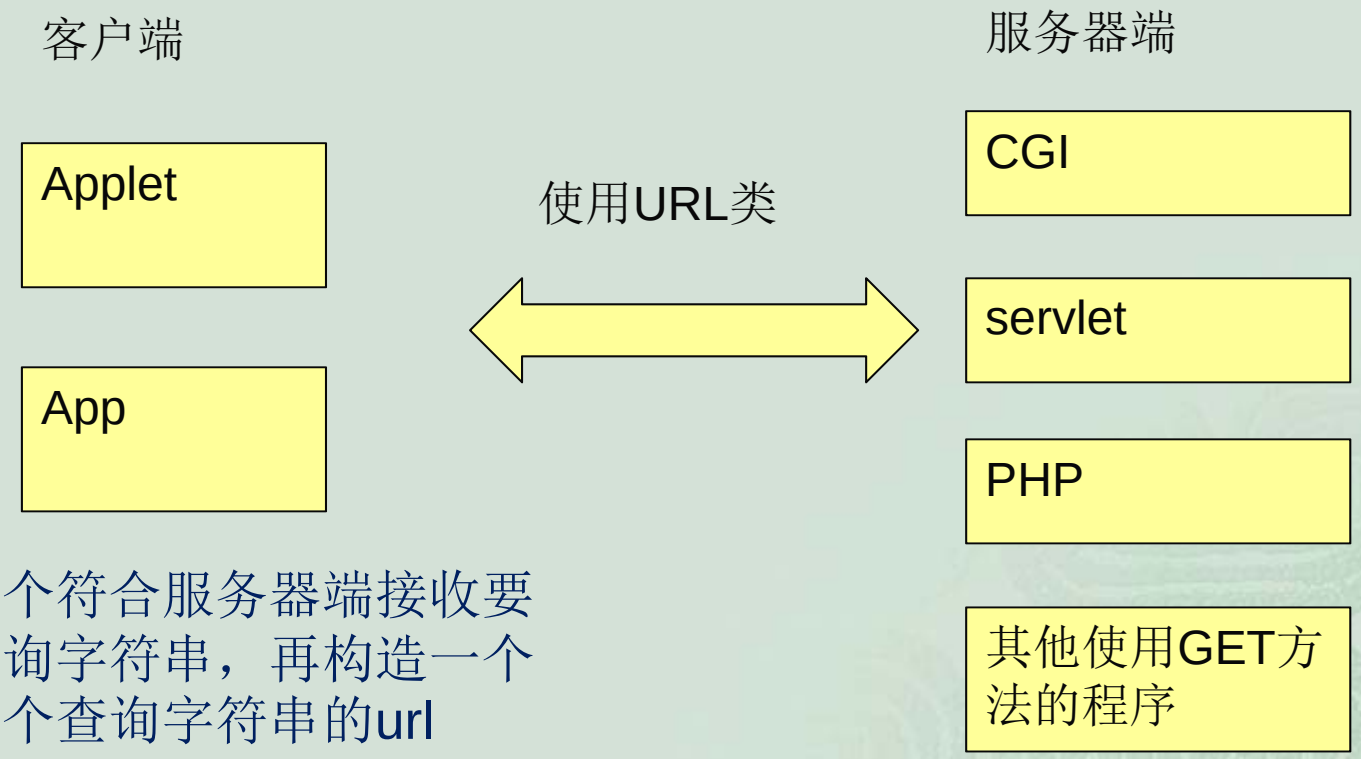
URLDecoder类

- `public static String decode(String s, String encoding) throws UnsupportedEncodingException`
- 解码，encode()的反变换，
- 对用x-www-form-url-encoded格式编码的字符串进行解码

思考

- 编程实现一代理服务器，限制深大局域网内的主机发起的访问百度搜索主页的请求，都过滤掉，不允许访问，给出访问受限提示。试思考探索编程实现。

5.访问服务器端的GET方法



1.若服务器端程序也是自己编写的，则客户端编程者也会知道服务器希望接收的名值对。
2.若服务器端使用了第三方程序，第三方程序文档也会指明它希望接收什么.
3.如果服务器端使用了第三方API, API文档会详细指用实现各种用途需发送什么数据.

GET方法适用场景

- GET方法仅限于安全的操作
  - 如:搜索请求、页面导航、设置书签、缓存、搜索、预取等
- 不能用于创建或修改资源等不安全操作
  - 如：在页面上发布一条评论、订购一个比萨等

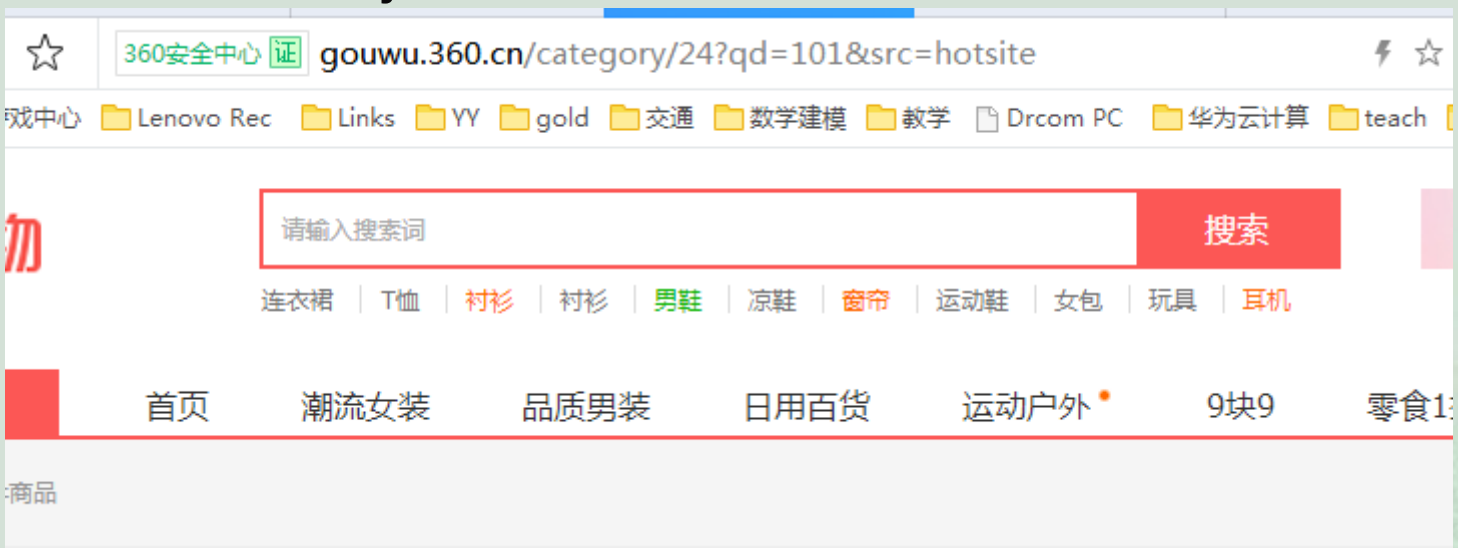
网页表单输入的处理

- 互联网应用程序常需要处理表单输入。
  - 1.编程时需先弄清楚程序希望得到什么输入
  - 2.创建一个查询字符串，包括必要的名值对输入
  - 3.构造一个包含此查询串的URL,打开URL连接，读取得到输入流

- 例如：一个360购物网站的表单处理程序如何处理搜索访问表单？https://gouwu.360.cn/
- https://www.dangdang.com/(当当网的搜索表单处理)

网页表单输入的处理

- 360购物网页　　Dmoz.java



- 对应html表单

```
      <form class="search-form" method="get"
action="http://gouwu.360.cn/list?qd=101" data-bk="search"
target="_blank">
                  <input type="text" name="q" value="家电"
id="search_keywords" placeholder="请输入搜索词"
autocomplete="off">
                <input type="hidden" name="src" value="form">
        <input type="hidden" name="qd" value="101">
<button class="search_btn" type="submit" id="searchSubmit">搜索
</button>
      </form>
```

DMoz.java

```java
2 import java.io.*;
3 import java.net.*;
4 public class DMoz {
5   public static void main(String[] args) {
6     String target = "";
7     for (int i = 0; i < args.length; i++) {
8       target += args[i] + " ";
9     }
10    target = target.trim();
11    QueryString query = new QueryString();
12    query.add("q", target);
13    try {
14      URL u = new URL("https://gouwu.360.cn/list?" + query);
15      try (InputStream in = new BufferedInputStream(u.openStream())) {
16        InputStreamReader theHTML = new InputStreamReader(in);
17        int c;
18        while ((c = theHTML.read()) != -1) {
19          System.out.print((char) c);
20        }
21      }
22    } catch (MalformedURLException ex) {
23      System.err.println(ex);
24    } catch (IOException ex) {
25      System.err.println(ex);
26    }
27  }
28 }
```

QueryString.java

```java
3  import java.io.UnsupportedEncodingException;
4  import java.net.URLEncoder;
5
6  public class QueryString {
7
8    private StringBuilder query = new StringBuilder();
9
10   public QueryString() {
11   }
12
13   public synchronized void add(String name, String value) {
14     query.append('&');
15     encode(name, value);
16   }
17
18   private synchronized void encode(String name, String value) {
19     try {
20       query.append(URLEncoder.encode(name, "UTF-8"));
21       query.append('=');
22       query.append(URLEncoder.encode(value, "UTF-8"));
23     } catch (UnsupportedEncodingException ex) {
24       throw new RuntimeException("Broken VM does not support UTF-8");
25     }
26   }
27
```

QueryString.java

```
27
28    public synchronized String getQuery() {
29      return query.toString();
30    }
31
32    @Override
33    public String toString() {
34      return getQuery();
35    }
36  }
37
```

Console

<terminated> DMoz [Java Application] C:\Program Files\Java\jdk1.8.0_152\bin\javaw.exe (2021年4月21日 上午11:16:10)

```
                <i class="icon icon-tmall"></i>
            <span class="sale-count">宸插敭18693</span>        <span class="address'
        <div class="look"><i class="icon icon-shop"></i>鍘荈涑鏞?</div>
</a>
  </li>


<a href="https://gouwu.360.cn/detail?id=72914392" target="_blank" data-text="瀵瑰
  <div class="img-outer"><img src="https://img.alicdn.com/tfscom/i4/1598428748/TB1
  <div class="desc clearfix">
                <h3 class="now-price"><em class="rmb">楼</em>22.80</h3>
            <span class="old-price">楼39.90</span>      </div>
  <p class="title">瀵瑰瀵寮?閹ㄑ暘绠辩洊甯冒鍙閹ㄑ暘绠辩僵宸惧簪澶存燧閲插皩缃十赕鎱烘   楹绒粴绑掇磕琛ｆ滿缃?<
  <p class="clearfix">
                <i class="icon icon-tmall"></i>
            <span class="sale-count">宸插敭1699</span>        <span class="address">
        <div class="look"><i class="icon icon-shop"></i>鍘荈涑鏞?</div>
</a>
<div class="loading"></div>  </li>
```

启动Dmoz时，设置运行参数为 洗衣机

程序问题与改进

- 编程思考问题：
- 360购物网站表单处理程序的返回结果为何有乱码？
- 请改进DMoz.java程序，改进后的程序可以正常显示查询结果，不会出现乱码。

6.Authenticator类

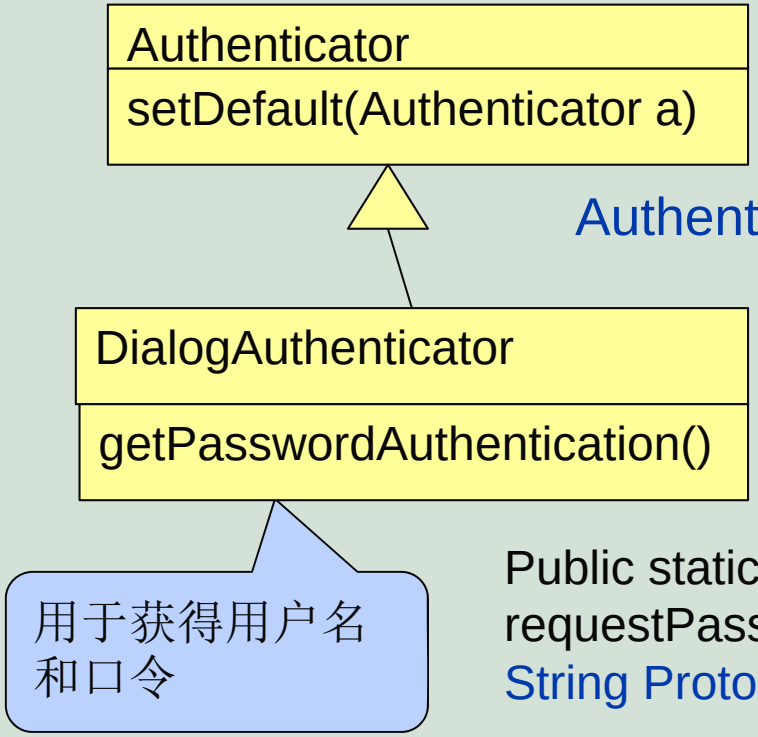
- 编程访问受口令保护的网站
  - 基于HTTP的标准认证
  - Session认证
  - 基于Cookie的非标准认证

Java的URL类可以访问使用HTTP认证的网站.

Java.net.Authenticator类

- 它为使用HTTP认证自我保护的网站提供用户名和口令

Public abstract class Authenticator extends Object



为使URL类可以使用获得用户名和口令的子类,必须先安装认证程序:

Authenticator.setDefault(new DialogAuthenticator());

当URL类需要用用户名和口令时,使用下面的静态方法询问获得:

Public static PasswordAuthentication requestPasswordAuthentication(InetAddress address, int port, String Protocol, String prompt, String scheme) throws …

PasswordAuthenticator类

- 它为授权者封装和保存用户名和口令

```
public final class PasswordAuthentication
extends Object
```

Constructors

Constructor and Description
`PasswordAuthentication(String userName, String password)` Initialize a new PasswordAuthentication

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method and Description
`String`	`getPassword()`
`String`	`getUserName()`

JPasswordField类

- 用以稍安全的方式询问用户口令的Swing组件

  javax.swing.JPasswordField

- 用户输入的内容会回显为*号，把口令存储为char数组，用完后可用0覆盖

  Public char[ ] getPassword( )

■ 自定义Authentication子类编程示例：

□ 自定义一个基于Swing的Authentication子类，它弹出一个对话框，向用户询问用户名和口令。其中，JPasswordField收集口令，JTextField获得用户名。

源码参考：DialogAuthenticaotr.java

SecureSourceViewer类：下载由口令保护的web页面的程序

DialogAuthenticator.java

```java
3  import java.awt.*;
4  import java.awt.event.*;
5  import java.net.*;
6  import javax.swing.*;
7  public class DialogAuthenticator extends Authenticator {
8    private JDialog passwordDialog;
9    private JTextField usernameField = new JTextField(20);
10   private JPasswordField passwordField = new JPasswordField(20);
11   private JButton okButton = new JButton("OK");
12   private JButton cancelButton = new JButton("Cancel");
13   private JLabel mainLabel
14       = new JLabel("Please enter username and password: ");
15   public DialogAuthenticator() {
16     this("", new JFrame());
17   }
18
19   public DialogAuthenticator(String username) {
20     this(username, new JFrame());
21   }
22
23   public DialogAuthenticator(JFrame parent) {
24     this("", parent);
25   }
26
```

DialogAuthenticator.java

```java
27  public DialogAuthenticator(String username, JFrame parent) {
28    this.passwordDialog = new JDialog(parent, true);
29    Container pane = passwordDialog.getContentPane();
30    pane.setLayout(new GridLayout(4, 1));
31
32    JLabel userLabel = new JLabel("Username: ");
33    JLabel passwordLabel = new JLabel("Password: ");
34    pane.add(mainLabel);
35    JPanel p2 = new JPanel();
36    p2.add(userLabel);
37    p2.add(usernameField);
38    usernameField.setText(username);
39    pane.add(p2);
40    JPanel p3 = new JPanel();
41    p3.add(passwordLabel);
42    p3.add(passwordField);
43    pane.add(p3);
44    JPanel p4 = new JPanel();
45    p4.add(okButton);
46    p4.add(cancelButton);
47    pane.add(p4);
48    passwordDialog.pack();
49
```

DialogAuthenticator.java

```java
    ActionListener al = new OKResponse();
    okButton.addActionListener(al);
    usernameField.addActionListener(al);
    passwordField.addActionListener(al);
    cancelButton.addActionListener(new CancelResponse());
  }

  private void show() {
    String prompt = this.getRequestingPrompt();
    if (prompt == null) {
      String site     = this.getRequestingSite().getHostName();
      String protocol = this.getRequestingProtocol();
      int    port     = this.getRequestingPort();
      if (site != null & protocol != null) {
        prompt = protocol + "://" + site;
        if (port > 0) prompt += ":" + port;
      } else {
        prompt = "";
      }
    }

    mainLabel.setText("Please enter username and password for "
        + prompt + ": ");
    passwordDialog.pack();
    passwordDialog.setVisible(true);
  }
```

DialogAuthenticator.java

```java
77    PasswordAuthentication response = null;
78    class OKResponse implements ActionListener {
79      @Override
80      public void actionPerformed(ActionEvent e) {
81        passwordDialog.setVisible(false);
82        // The password is returned as an array of
83        // chars for security reasons.
84        char[] password = passwordField.getPassword();
85        String username = usernameField.getText();
86        passwordField.setText("");// Erase the password in case this is used again.
87        response = new PasswordAuthentication(username, password);
88      }
89    }
90    class CancelResponse implements ActionListener {
91      @Override
92      public void actionPerformed(ActionEvent e) {
93        passwordDialog.setVisible(false);
94        // Erase the password in case this is used again.
95        passwordField.setText("");
96        response = null;
97      }
98    }
99    public PasswordAuthentication getPasswordAuthentication() {
100     this.show();
101     return this.response;
102   }
103 }
```

SecureSourceViewer.java

```java
5  public class SecureSourceViewer {
6    public static void main (String args[]) {
7      Authenticator.setDefault(new DialogAuthenticator());
8      for (int i = 0; i < args.length; i++) {
9        try {
10         // Open the URL for reading
11         URL u = new URL(args[i]);
12         try (InputStream in = new BufferedInputStream(u.openStream())) {
13           // chain the InputStream to a Reader
14           Reader r = new InputStreamReader(in);
15           int c;
16           while ((c = r.read()) != -1) {
17             System.out.print((char) c);
18           }
19         }
20       } catch (MalformedURLException ex) {
21         System.err.println(args[0] + " is not a parseable URL");
22       } catch (IOException ex) {
23         System.err.println(ex);
24       }
25       // print a blank line to separate pages
26       System.out.println();
27     }
28     // Since we used the AWT, we have to explicitly exit.
29     System.exit(0);
30   }
31 }
```

编程思考

- 如何编程实现网页加密，使某个或某些特定网页的查看受口令保护？
- http://www.silsea.com/ip/login.php

谢谢